

第二次作业反馈

林洛昊

2024.9

1 相关知识回顾

这周的课堂和作业主要还是线性表尤其是顺序表相关的内容，我在此列举一下 ppt 上的相关内容首先是定义：

```
typedef int ElemType; // 元素类型
typedef struct {
    ElemType *elem; // 基地址
    int listsize;    // 当前分配内存大小，单位是sizeof(ElemType)
    int length;      // 当前线性表长度
} SqList;
```

然后是 ppt 上给出的一些基本操作的算法实现，这些函数大家在写作业的时候可以放心用：

```
void InitList_sq(SqList &L, int msize) {
    L.elem = new ElemType[msize];
    L.listsize = msize;
    L.length = 0;
}

void DestroyList_sq(SqList &L) {
    delete []L.elem;
    L.length = 0;
    L.listsize = 0;
}

bool ListEmpty_sq(SqList L) {
    return (L.length == 0);
}

int ListLength_sq(SqList L) {
    return L.length;
}

void GetElem_sq(SqList L, int i, ElemType &e) {
    if (i < 1 || i > L.length) ErrorMsg("Invalid i value");
    e = L.elem[i-1];
}

int LocateElem_sq(SqList L, ElemType e) {
    for (int i = 1; i <= L.length; ++i)
        if (L.elem[i-1] == e) return i;
    return 0;
}
```

```

}
void Increment(SqList &L, int inc_size) {
    ElemType *a = new ElemType[L.listsize + inc_size];
    for (int i=0; i<L.length; ++i) a[i] = L.elem[i];
    delete []L.elem; L.elem = a;
    L.listsize += inc_size;
}

```

细心的同学可能会发现，在我们定义不同的函数时，有时候会用 SqList L，有时候会用 SqList &L，这是为什么呢？我这里讲一下我自己的理解，因为主函数里定义的 L 和我们在函数里设定的参数 L 实际上不是同一个，所以如果我们希望修改主函数里的 L 的话，就需要给参数 L 加上引用符 &，让它引用主函数中 L 的地址。总结一下就是，如果你要改动这个数据，就加 &，否则不用，这一点在我们这次的作业也有所体现。

2 作业题目解答

2.3 已知顺序表 La 中数据元素按非递减有序排列，试写一个算法，将元素 x 插到 La 的合适位置上，保持该表的有序性

这题的算法其实 ppt 上就有啦，等于白给，考试可不会有这么好的事情嘛

```

void ListInsert_sq(SqList &L, int i, ElemType e) {
    if (i<1 || i>L.length+1) ErrorMsg("Invalid i value");
    if (L.length == L.listsize) Increment(L, 10);
    for (int j=L.length-1; j>=i-1; --j) L.elem[j+1] = L.elem[j];
    L.elem[i-1] = e;
    ++L.length;
}

```

不过我们管中窥豹一下，还是可以从这题看出我们写代码时要注意的几个点：检查输入是否合法；考虑极限边缘情况——我相信这也是大家写代码常犯的错误之一。

此外，有同学自己定义了个 swap 函数，我这里稍微列举两种 swap 的写法供大家参考，大家注意一下用法上的区别

```

void swap1(int *p1, int *p2)
{
    int temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}

void swap2(int &x, int &y)
{
    int temp=x;
    x = y;
    y = temp;
}

int main()
{
    int a=127, b=343;

```

```

        swap1(&a,&b);
        swap2(a,b);
        return 0;
}

```

2.5 试写一个算法，实现顺序表的就地逆置，即在原表的存储空间将线性表 $(a_1, a_2, \dots, a_{n-1}, a_n)$ 逆置为 $(a_n, a_{n-1}, \dots, a_2, a_1)$

这题也比较简单，很直观的一个思路是头尾互换，各往内缩一步之后再互换，不过需要注意的是题目说的是就地逆置，也就是辅助空间的复杂度为 $O(1)$ ，所以新建一个线性表来存储的方法是不可以的喵！

```

void Element_Invert(Sqlist &L)
{
    int len=L.length;
    Elemtype tmp;
    int i;
    for(i=0;i<len/2;i++)
    {
        tmp=L.elem[i];
        L.elem[i]=L.elem[len-i-1];
        L.elem[len-i-1]=tmp;
    }
}

```

2.8 已知线性表用顺序存储结构表示，表中数据元素为 n 个正整数，试写一个算法，分离该表中的奇数和偶数，使得所有奇数集中放在左侧，偶数集中放在右侧，要求不借用辅助数组且时间复杂度为 $O(n)$

这题应该是本周作业里比较有难度的一道了（虽然也不是很难），既然奇数在左偶数在右，一个直观的想法是我们用两个“哨兵”分别从左往右和从右往左数，使用哨兵的好处是我们可以简化边界条件的处理，只要保证左哨兵 $<$ 右哨兵即可，这种使用哨兵的思想我们后续也会在插入排序的优化中使用到。

```

void Element_EvenOdd(Sqlist &L)
{
    int left=0,right=L.length-1,tmp;
    while(left<right)
    {
        while(L.elem[left]%2==1) //为奇数
        {
            left++;
        }
        while(L.elem[right]%2==0) //为偶数
        {
            right--;
        }
        tmp=L.elem[left];
        L.elem[left]=L.elem[right];
        L.elem[right]=tmp;
    }
}

```

这题有的同学把% 写成/了，要注意求余符号和除号的区别，大家可以留意到三道作业题的参数都是 SqList &L，这正是因为我们要修改主函数的线性表呀

3 补充阅读资料

Hello 算法-第四章数组与链表

4 总结

虽然线性表（尤其是顺序表）由于考起来没有链表难，在考试和大作业中出现的频率可能不如链表，不过作为最基础最直观的一个数据结构，大家还是有必要掌握其一些基础操作的思想的。我在批改大家作业的时候还留意到，可能是不太适应手写代码，有的在写代码的时候可能会出现漏了右大括号、没定义一个变量就直接使用的问题，大家书写代码的时候还是稍微注意一下这点，毕竟考试肯定是要手写的（无奈）。